

Trabajo Final · Asignatura Big Data

Caso: Modelo Energético Aragón — predicción del consumo del
parque edificado no certificado

192.145 certificados · arquitectura Medallion sobre Databricks

PRESENTADO POR

Valentina Mesa Álvarez

Ricardo Builes Escobar

DOCENTE

Yeis Livis Taborda Henao

MÓDULO

Big Data

FECHA

Medellín, septiembre de 2026

Contenido

01 El problema

02 Los datos

03 Arquitectura

04 El pipeline de ingesta

05 Calidad del dato

06 El modelo y su validación

07 Fuga de datos

08 Consumo y visualización

09 Decisiones y correcciones

10 Limitaciones

11 Reproducibilidad

El problema

En España, vender o alquilar un inmueble exige un certificado de eficiencia energética: lo emite un técnico habilitado, vale diez años y se registra en el organismo de la comunidad autónoma (RD 390/2021). De ahí sale el registro de Aragón, y de ahí su primera particularidad: **no es un censo del parque edificado, sino de trámites**. Solo aparece el inmueble que en algún momento se vendió o se alquiló; lo que nunca ha rotado —probablemente también lo peor conservado— falta.

Mientras tanto, la Directiva europea 2024/1275 obliga a reducir un 16 % el consumo de energía primaria del parque residencial antes de 2030, y exige que al menos el 55 % de esa reducción venga de rehabilitar los edificios peor calificados. Para cumplirlo hay que identificarlos, y certificar puerta a puerta no escala.

Nuestra pregunta: ¿qué consumo cabe esperar de un inmueble sin certificar, usando solo los datos que se conocen sin visitarlo?

El destinatario es la Dirección General de Energía del Gobierno de Aragón, que gestiona el registro y decide a qué municipios dirigir las ayudas a la rehabilitación.

Este informe documenta la construcción de la solución: el pipeline de datos, las reglas de calidad aplicadas y la validación del modelo. El análisis del parque edificado aragonés queda fuera del alcance — el objeto del trabajo es la ingeniería y el modelo, no las conclusiones territoriales.

Los datos

Fuente: Registro de Certificación de Eficiencia Energética de Edificios de Aragón, publicado por el Gobierno de Aragón y accesible a través de Kaggle. El fichero original contiene **192.145 registros**, una fila por certificado emitido.

Cada fila es un trámite administrativo, no una medición instrumental. Eso explica buena parte de lo que encontramos después: errores de tecleo, duplicados y valores centinela son lo esperable en un formulario que rellena una persona.

Columnas del origen

COLUMNA	CONTENIDO
numcert	Número de certificado. Se repite: no sirve como clave
fec_emision · fec_expira	Emisión y caducidad. La validez legal es de diez años
consumo_ener	Variable objetivo. Energía primaria no renovable, kWh/m²·año
emision_co	Emisiones, kgCO ₂ /m ² ·año
clasificacion_consumo	Letra A–G derivada del consumo
clasificacion_emisiones	Letra A–G derivada de las emisiones
tipoedi · estadoedi	Tipo y estado del edificio
anio	Año de construcción
superficie	Superficie útil, m ²
munic · prov	Municipio y provincia
refcatastral	Referencia catastral del inmueble
direccion	Dirección postal
coordenadas	Coordenadas UTM, ambas componentes en un solo campo

Tres observaciones que condicionaron el diseño

El consumo y las emisiones son la misma medida. Las emisiones resultan de multiplicar el consumo por el factor de emisión del combustible. No son un dato independiente, y por eso quedaron fuera del modelo (sección 7).

Las letras A–G no son umbrales absolutos. Se calculan contra un edificio de referencia de la misma zona climática y uso, de modo que la misma letra corresponde a consumos distintos según dónde esté el edificio.

El dataset no trae la zona climática, que es justamente la variable que explica esa diferencia. Hubo que derivarla a partir de la provincia y el municipio.

SECCIÓN 03

Arquitectura

La solución sigue una **arquitectura Medallion** sobre Databricks, con Delta Lake como formato de almacenamiento y Unity Catalog como capa de gobierno.

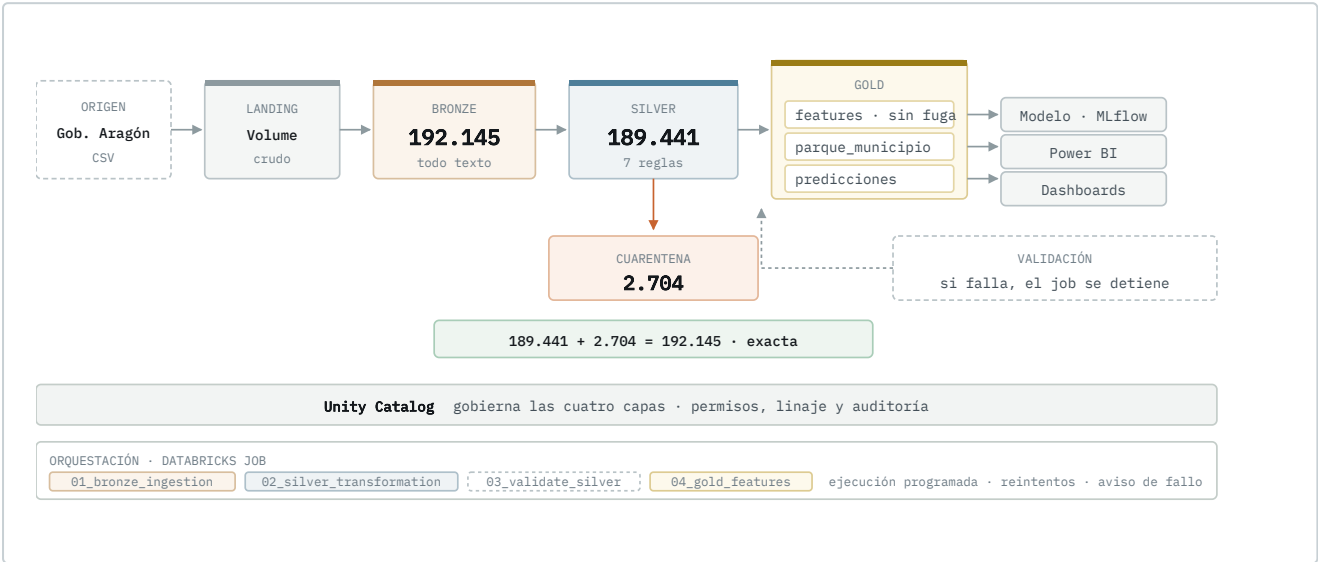


Figura 3.1 — Arquitectura Medallion. Del origen a las capas de consumo, con la cuarentena como rama explícita y la validación cortando el flujo antes de Gold.

Por qué tres capas y no una

Si se limpiara el CSV en una sola pasada y una regla resultara mal calibrada, el original ya no existiría y habría que volver a descargarlo. Nos pasó: la regla de valores atípicos del consumo estaba mal y hubo que rehacer Silver entera. Como Bronze conserva el crudo intacto, bastó con reejecutar el pipeline (sección 9).

Responsabilidad de cada capa

CAPA	QUÉ GARANTIZA	ESCRITURA
Bronze	El dato llega intacto. Todo se lee como texto, sin inferir tipos	Añadir. Solo crece
Silver	El dato es confiable. Reglas declaradas y cuarentena auditable	Sobrescribir
Gold	El dato está preparado para un consumidor concreto	Sobrescribir

Esa asimetría es deliberada: Bronze es la fuente de la verdad y solo crece; Silver y Gold son derivadas y se regeneran cada vez que cambia una regla. Es lo que hace el pipeline reproducible.

Decisiones de diseño

DECISIÓN	ALTERNATIVA DESCARTADA	MOTIVO
Bronze lee todo como texto	Inferir tipos al leer	Un valor con coma decimal española pasa a nulo sin aviso. El dato se pierde en silencio
Filas rechazadas a cuarentena	Descartarlas con un filtro	Permite auditar cada descarte y medir cuántas filas costó cada regla
Delta Lake	Parquet o CSV en carpetas	Transacciones ACID: si el proceso se cae a mitad de escritura, la tabla queda en su última versión buena
Volume para el archivo crudo	Leer directamente de la URL	El crudo queda archivado y cualquier Bronze pasado se puede reconstruir
ELT	ETL clásico	Cargar primero y transformar después permite corregir una regla sin volver al origen

El pipeline de ingesta

Capa cruda

El CSV se deposita en un volume de Unity Catalog y de ahí se lee a Bronze. La regla es una sola: **no se transforma nada**. Todo se lee como texto, sin inferir tipos.

Esa decisión evita un fallo silencioso. Si Spark infiere los tipos, un valor con coma decimal española no se puede convertir y **pasa a nulo sin dar error**: el dato existía en el archivo y desaparece de la tabla. Leyéndolo como texto llega intacto, y es la capa siguiente la que lo rechaza dejando constancia.

A cada fila se le añade su linaje: identificador de lote, archivo de origen, huella SHA-256 e instante de carga. Resultado: **192.145 filas** en `cee_aragonv2.bronze.certificados_raw`.

Idempotencia

El origen publica el fichero completo cada vez. Con carga por adición, cada reejecución duplicaría las filas.

No sirve combinar por clave porque **el número de certificado se repite** en este dataset. Se resolvió llevando control de los lotes ya procesados mediante la huella del archivo. La comprobación va como aserción, no como mensaje impreso: dentro de una tarea programada un mensaje no detiene nada.

antes

192.145

después

192.145

← tras reejecutar el notebook completo

Orquestación

#	TAREA	QUÉ HACE
01	bronze_ingestion	Volume → capa cruda con linaje
02	silver_transformation	Tipado, reglas de calidad, cuarentena
03	validate_silver	Comprueba invariantes. Si falla, corta el job
04	gold_features	Tablas de consumo

La validación va **antes** de Gold. Si Silver incumpliera una invariante, el job se detiene y las tablas de consumo conservan su última versión buena en lugar de actualizarse con datos malos.

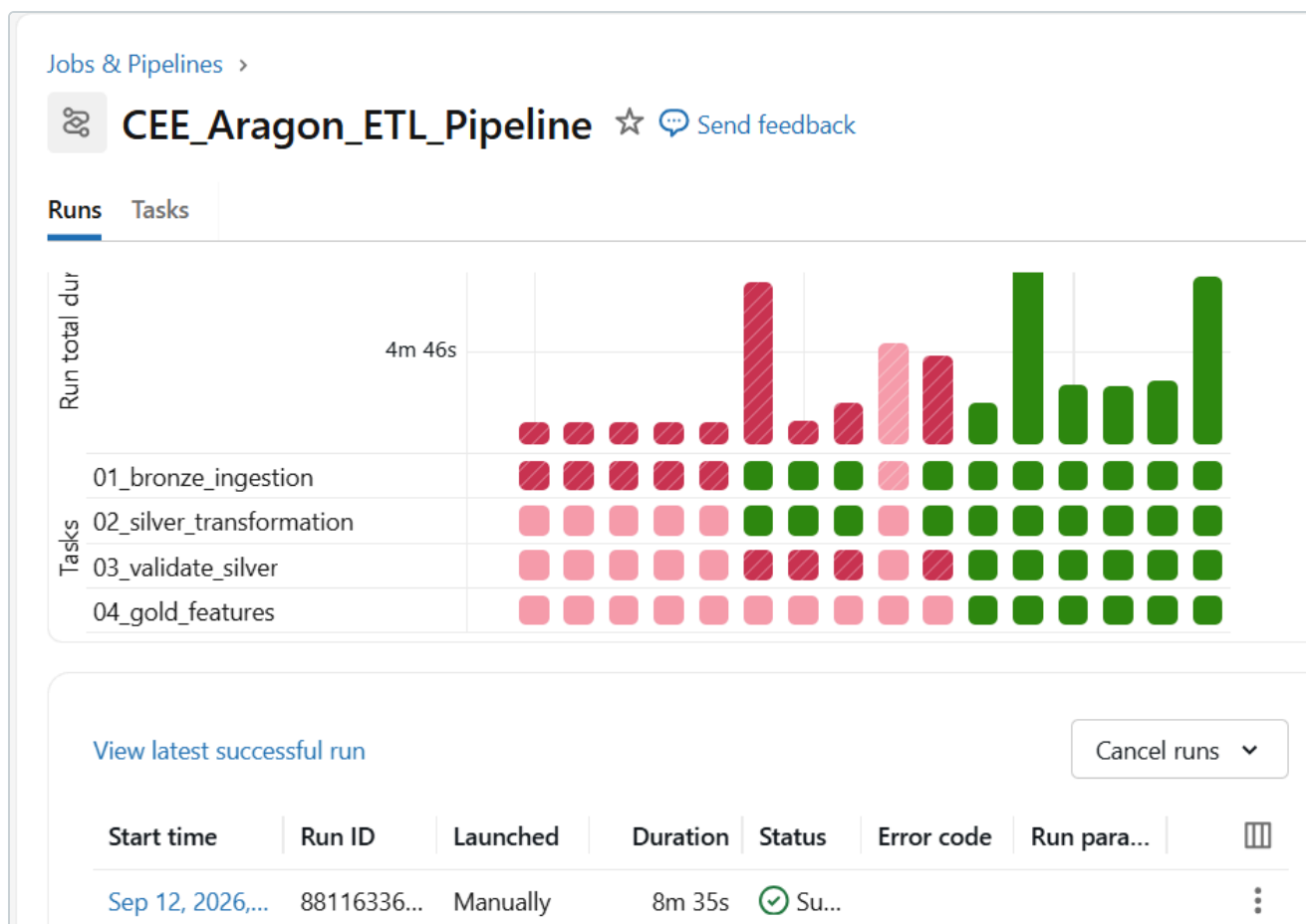


Figura 4.1 — Historial del job `CEE_Aragon_ETL_Pipeline`. Cada columna es una ejecución completa y cada fila una tarea. El historial se lee de izquierda a derecha: las ejecuciones fallidas corresponden al proceso de depuración que documenta la sección 9, y las cuatro tareas en verde son el estado final.

La captura muestra algo que conviene señalar: **la tarea 03 falla antes que la 04 en varias ejecuciones**. Es exactamente el comportamiento que se buscaba al colocar la validación delante de Gold — cuando Silver no cumplía una invariante, las tablas de consumo no llegaron a reescribirse.

Calidad del dato

Las filas que incumplen una regla **no se eliminan**: se escriben en una tabla aparte junto con el identificador de la regla que las rechazó. Eso permite auditar cada descarte, medir cuánto costó cada decisión y recalcular la limpieza sin volver al origen.

Las reglas aplicadas

La capa Silver declara las reglas como una estructura de datos al principio del notebook, no dispersas por el código. Cada una lleva identificador, columna afectada, descripción legible y condición:

- **R01** — año de construcción dentro de [1800, 2024]
- **R02** — año de emisión del certificado dentro de rango
- **R03** — superficie menor o igual a 50.000 m²
- **R04** — emisión de CO₂ dentro de rango físico
- **R05** — emisión de CO₂ convertible a número
- **R06** — consumo dentro de [20, 1000] kWh/m²·año
- **R08** — consumo convertible a número

Una fila puede incumplir varias reglas a la vez, y se registran todas, no solo la primera. Por eso en el desglose aparecen combinaciones como `R05 + R06`.

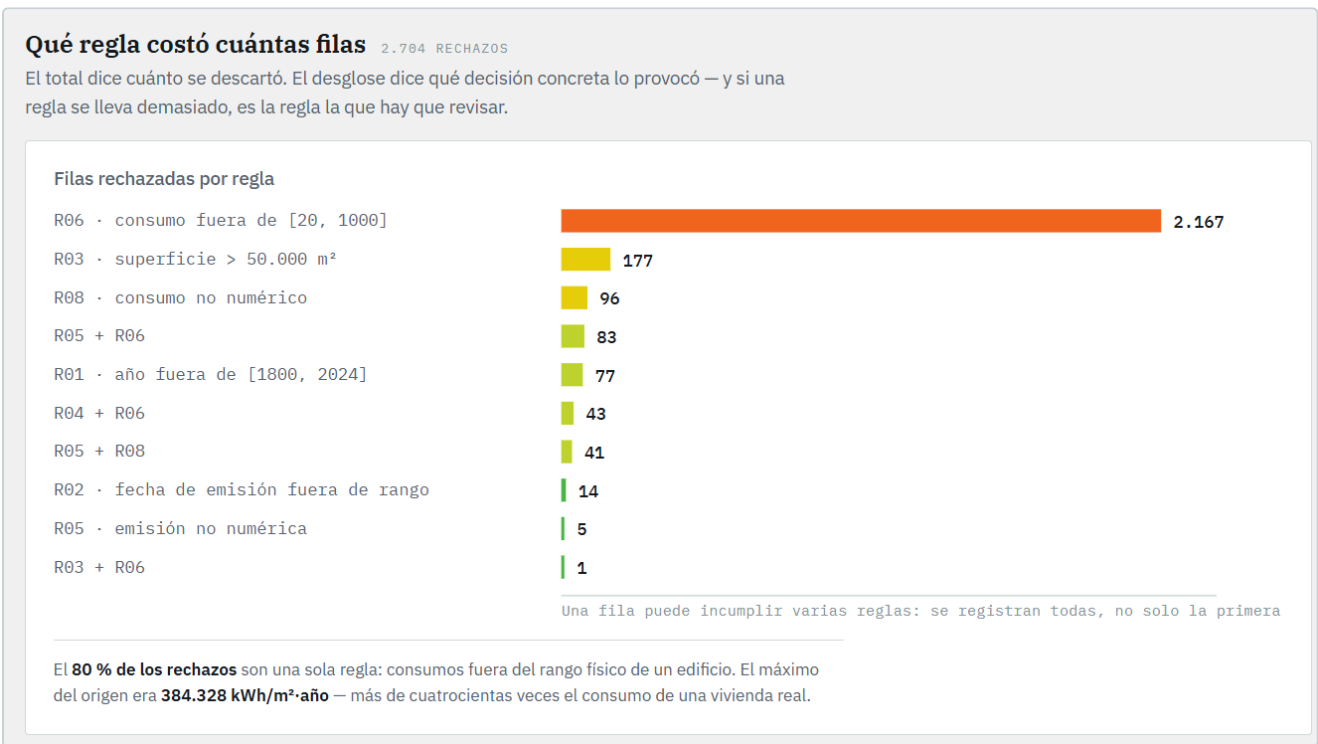


Figura 5.1 — Filas rechazadas por regla. El 80 % de los rechazos corresponde a una sola: consumos fuera del rango físico de un edificio.

Reconciliación

189.441 válidas + 2.704 en cuarentena = 192.145 de origen

La comprobación se ejecuta como aserción dentro del pipeline: si la suma no cuadra, el job se detiene y las tablas de consumo no se escriben. Es la prueba más corta de que la ingesta no pierde datos.



Figura 5.2 — Indicadores de calidad: filas de origen, válidas, en cuarentena y resultado de la reconciliación.

Dos decisiones de criterio

El umbral del consumo viene del dominio, no de una fórmula. Una vivienda real consume entre 20 y 600 kWh/m²·año; ampliamos el techo a 1.000 para admitir edificios de otros usos, que legítimamente consumen más. Un criterio automático basado en percentiles habría dejado pasar los valores imposibles, porque el propio percentil queda contaminado por ellos.

Los 76 inmuebles sin letra se conservan en Silver. El origen codifica la ausencia de calificación con un guion. Un valor centinela no es una categoría: se normaliza a nulo. Esas filas mantienen el consumo —que es la variable objetivo— y solo les falta una etiqueta derivada, de modo que apartarlas sería descartar observaciones válidas.

El modelo y su validación

Qué se predice, y con qué

El objetivo es el **consumo de energía primaria no renovable en kWh/m²·año** de un inmueble que todavía no tiene certificado. Ese escenario impone el criterio para admitir variables: *¿existiría este dato antes de que un técnico visite el inmueble?*

VARIABLE	ORIGEN DEL DATO
Año de construcción	Catastro
Superficie útil	Catastro
Tipo de edificio	Catastro
Provincia y zona climática	Dirección del inmueble

El conjunto de entrenamiento son **151.640 inmuebles** y el de prueba **37.801**, apartados desde el principio y que el modelo nunca vio al entrenar.

La línea base

Antes de entrenar nada se calcula la predicción más simple posible: el consumo medio de cada zona climática. Sin esa referencia, ninguna métrica del modelo significa nada.

Línea base – media por zona climática: MAE **88,09** kWh/m²·año

Comparación de algoritmos

MODELO	MAE	RMSE	R ²	MEJORA
Media por zona (<i>línea base</i>)	88,09	—	—	—
Regresión lineal	80,74	110,30	0,131	+8,3 %
Random forest	72,61	100,43	0,279	+17,6 %
Gradient boosting	71,71	99,42	0,294	+18,6 %

Los tres quedaron registrados en MLflow con sus parámetros, sus métricas y el MAE de la línea base como referencia en cada ejecución, de modo que las corridas son comparables entre sí.

Comparado contra la predicción trivial 4 MODELOS

Un error de 71,7 kWh no significa nada por sí solo. Significa algo comparado con lo que se acierta prediciendo, sin modelo, la media de consumo de cada zona climática.

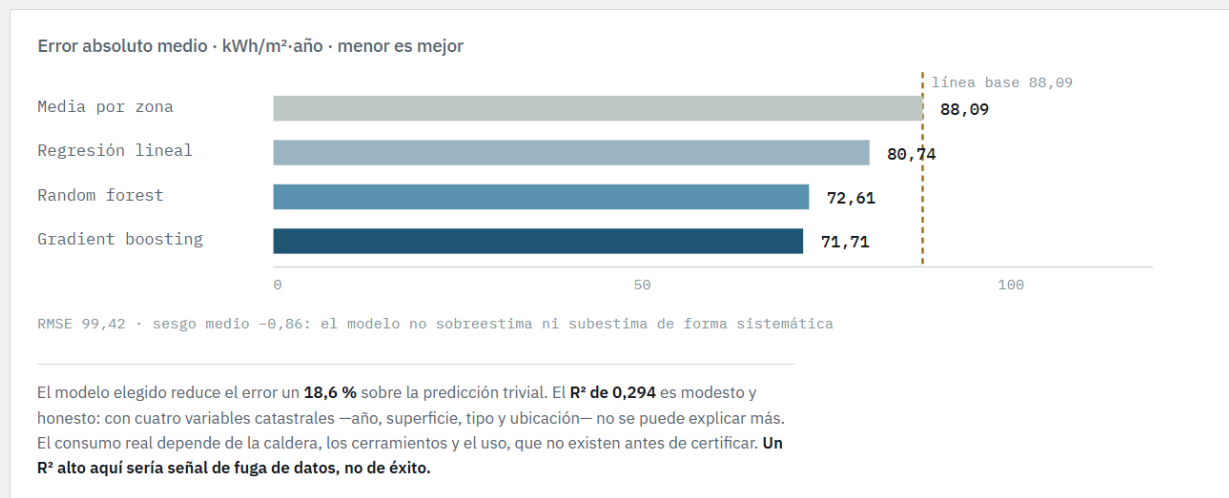


Figura 6.1 — Error absoluto medio de cada algoritmo frente a la línea base.

Qué significa cada cifra

MÉTRICA	VALOR	INTERPRETACIÓN
MAE	71,71	Al estimar el consumo de un inmueble el modelo se equivoca, en promedio, en 71,7 kWh/m²·año . Sobre un consumo medio de 256, es un error del 28 %
RMSE	99,42	Bastante mayor que el MAE: la mayoría de las predicciones son razonables, pero hay una minoría de inmuebles donde el error es grande
R ²	0,294	Explica el 29 % de la variación del consumo. El resto depende de la caldera, los cerramientos y el uso — datos que el catastro no registra
Sesgo medio	-0,86	Prácticamente cero sobre una media de 256: el modelo no sobreestima ni subestima de forma sistemática
Mejora	+18,6 %	Reduce el error casi una quinta parte frente a predecir la media de la zona

Clasificación derivada

La rúbrica pide matriz de confusión y curvas de modelo, que son de clasificación, mientras que el objetivo es continuo. Se resuelve sin cambiar de objetivo: el consumo predicho se convierte en una decisión binaria —*¿es ineficiente?*— usando un umbral.

Como la escala A–G es relativa a la zona climática, ese umbral **no puede ser un número único**. Se deriva de los propios datos, como el percentil 95 del consumo de los edificios calificados A–D de cada zona:

ZONA CLIMÁTICA	UMBRAL DE INEFICIENCIA
C3	333,8 kWh/m ² · año
D2	300,0
D3	284,8

La separación de 49 kWh entre el umbral más alto y el más bajo es la razón de que la zona climática entre al modelo como variable predictora.



Figura 6.2 — Matriz de confusión sobre los 37.801 inmuebles del conjunto de prueba, con las métricas de clasificación.

MÉTRICA	VALOR	INTERPRETACIÓN
Precisión	59,2 %	De cada 100 inmuebles que el modelo marca, 59 resultan realmente ineficientes
Recall	33,1 %	Encuentra uno de cada tres inmuebles ineficientes que existen
Exactitud	79,7 %	Aciertos totales sobre el conjunto de prueba
Modelo trivial	77,4 %	Decir siempre «eficiente» acertaría el 77,4 %. La exactitud del modelo solo supera eso en 2,3 puntos: la métrica engaña cuando una clase domina

Presentamos las dos últimas juntas deliberadamente. Reportar una exactitud del 79,7 % sin decir que el modelo trivial alcanza el 77,4 % daría una impresión falsa del desempeño.

Utilidad del modelo

La exactitud no es la métrica adecuada para este caso. La pregunta correcta es cuánto mejora el modelo la eficacia de una inspección respecto a elegir al azar:



Figura 6.3 — Proporción de inmuebles ineficientes encontrados inspeccionando al azar frente a inspeccionar los que marca el modelo.

El modelo no es un buen clasificador — es un buen **priorizador**. Con un recall del 33 % no sirve para encontrar todos los inmuebles ineficientes, pero **multiplica por 2,6** la proporción de aciertos frente a una selección aleatoria.

Rendimiento por zona

ZONA	EVALUADOS	MAE	PRECISIÓN	RECALL	LIFT
C3	29.688	69,3	60,4 %	25,1 %	3,09x
D2	5.746	79,6	58,6 %	55,0 %	1,67x
D3	2.367	82,5	55,4 %	37,3 %	1,80x

El desempeño no es homogéneo: el modelo rinde mejor donde tiene más datos. C3 concentra el 79 % del conjunto de prueba y es donde el error es menor. Esa asimetría desaconseja extrapolar el resultado global a las zonas con menos observaciones.

Fuga de datos

El dataset trae columnas que predicen el consumo casi a la perfección, y ninguna puede usarse.

Por qué las emisiones no son una variable

Las emisiones de CO₂ no son una medición independiente: resultan de **multiplicar el consumo por el factor de emisión del combustible**. Son el objetivo expresado en otra unidad. En el perfilado de la capa cruda, la correlación observada entre ambas columnas es prácticamente 1.

Pero hay una segunda razón, y es la que de verdad decide: **un inmueble sin certificar tampoco tiene emisiones medidas**. La columna no existe en el momento en que hace falta la predicción. Esa razón bastaría por sí sola, aunque la primera no fuera cierta.

Lo mismo ocurre con las dos columnas de clasificación A–G, que son el objetivo discretizado, y con las fechas de emisión y expiración, que solo existen cuando el certificado ya se emitió.

¿Existiría este dato antes de que un técnico visite el inmueble?

COLUMNA	DECISIÓN	MOTIVO
emision_co2	Excluida	Es el objetivo multiplicado por un factor
clasificacion_consumo	Excluida	Es el objetivo discretizado
clasificacion_emisiones	Excluida	Lo mismo, por la vía de las emisiones
fecha_emision · fecha_expiracion	Excluidas	Solo existen si ya hay certificado
numero_certificado · ref_catastral	Excluidas	Identificadores, no señal
estado_edificio	Excluida	Un solo valor en el 99 % de las filas

La selección se hace con **lista blanca explícita**, no descartando columnas. Si el origen añadiera un campo nuevo, con descarte entraría solo al modelo; con lista blanca no entra nada que no se haya decidido.

Dónde se resuelve

Silver **conserva** las emisiones y las letras: son datos verdaderos del certificado y hay consultas que las necesitan. Lo que se hace es no dejar que el modelo las vea, y eso ocurre al construir la tabla de entrenamiento en Gold.

La fuga de datos **no es un problema de limpieza, es un problema de diseño de la tabla de entrenamiento**. Si fuera de limpieza se resolvería borrando la columna en Silver. Como es de diseño, se resuelve en Gold decidiendo qué ve cada consumidor.

Por qué un R² de 0,294 es la respuesta correcta

Con las columnas anteriores dentro, el ajuste sería casi perfecto y el modelo no serviría para nada: estaría leyendo la respuesta. **Un R^2 alto en este problema es señal de fuga, no de éxito.**

El 29 % que se explica corresponde a lo que el catastro sabe de un edificio: cuándo se construyó, cuánto mide, de qué tipo es y dónde está. El 71 % restante depende de la instalación térmica, los cerramientos y el uso — información que, por definición, solo aparece cuando alguien va a verlo.

SECCIÓN 08

Consumo y visualización

Las tablas Gold se exponen por tres vías distintas, cada una para un uso diferente.

Dashboard nativo de Databricks

Construido sobre las tablas Gold dentro del propio Lakehouse, ejecutándose contra el SQL Warehouse. No requiere herramienta externa y se comparte con un enlace.

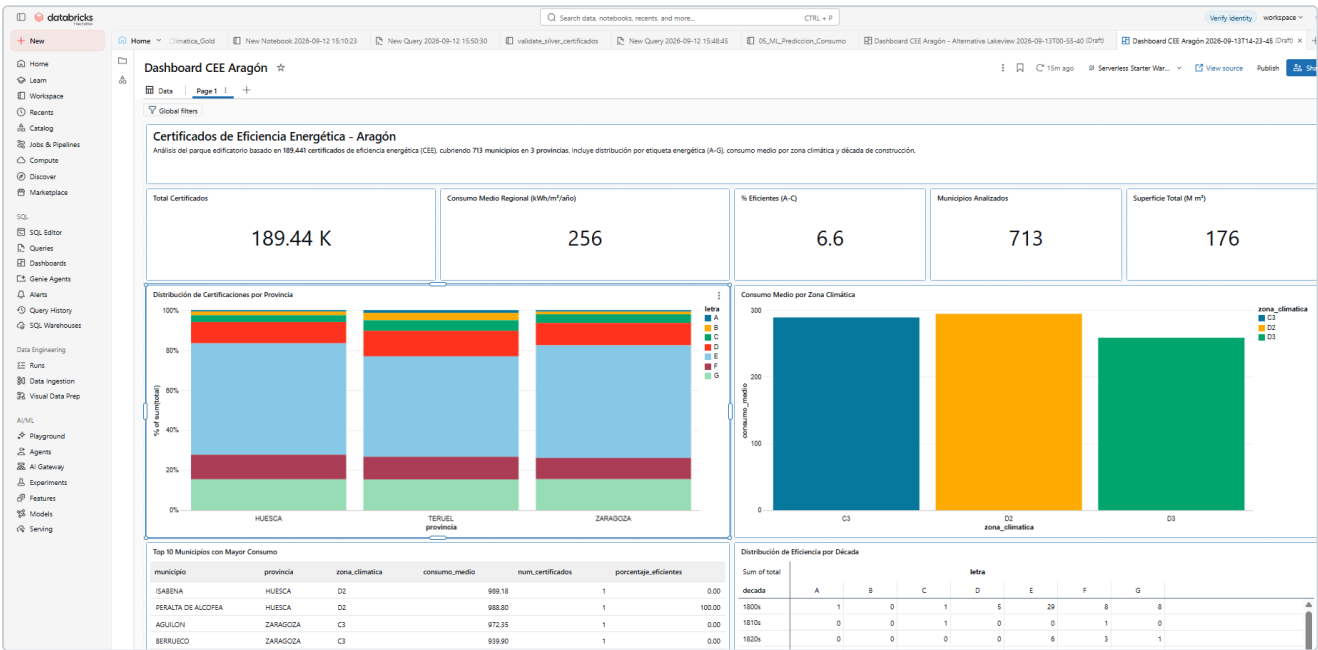


Figura 8.1 — Dashboard nativo de Databricks sobre las tablas Gold.

Power BI conectado al Lakehouse

El informe de Power BI se conecta **directamente contra el SQL Warehouse de Databricks**, sin exportar ni copiar los datos a un archivo intermedio. Las tablas del modelo son las mismas tablas de Unity Catalog: cuando el pipeline reescribe Silver o Gold, el informe consulta la versión nueva sin ningún paso manual.

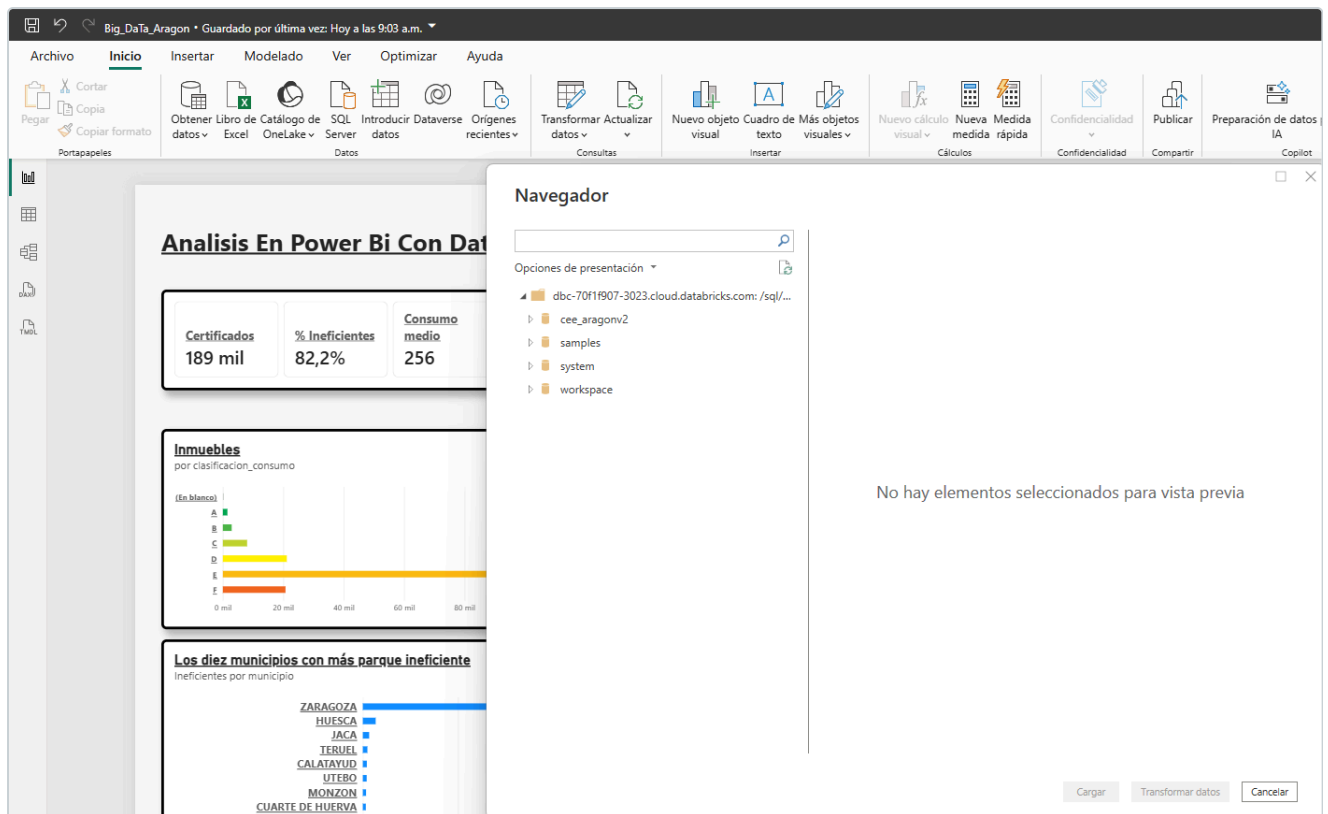


Figura 8.2 — Conexión de Power BI al SQL Warehouse de Databricks. El navegador muestra el catálogo `cee_aragonv2` junto a los del propio workspace: las tablas se leen del Lakehouse, no de un archivo exportado.

El modelo semántico reproduce el pipeline completo, no solo la capa final. Junto a las tablas de consumo (`features`, `parque_municipio`, `predicciones`) están también `certificados_raw`, `cuarentena` y `_control_batch`, de modo que **la trazabilidad y la reconciliación se pueden comprobar desde el propio informe**: cuántas filas entraron, cuántas se rechazaron y por qué regla.

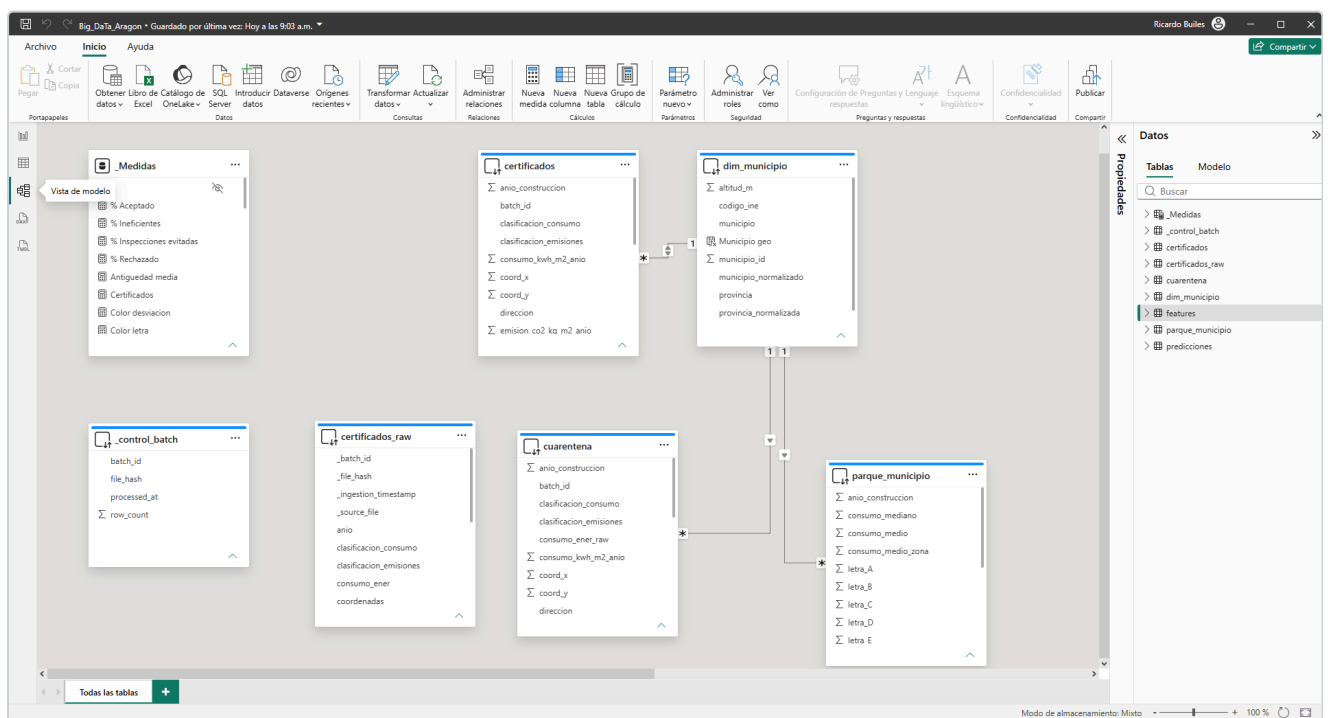


Figura 8.3 — Modelo semántico en Power BI. Las relaciones se apoyan en `dim_municipio` como dimensión compartida, y la tabla `_Medidas` agrupa los cálculos en DAX.

Las medidas se centralizan en una tabla `_Medidas`, separada de los datos, y cubren tanto negocio (certificados, antigüedad media, porcentaje de ineficientes) como calidad del pipeline (porcentaje aceptado y rechazado) y utilidad del modelo (inspecciones evitadas).

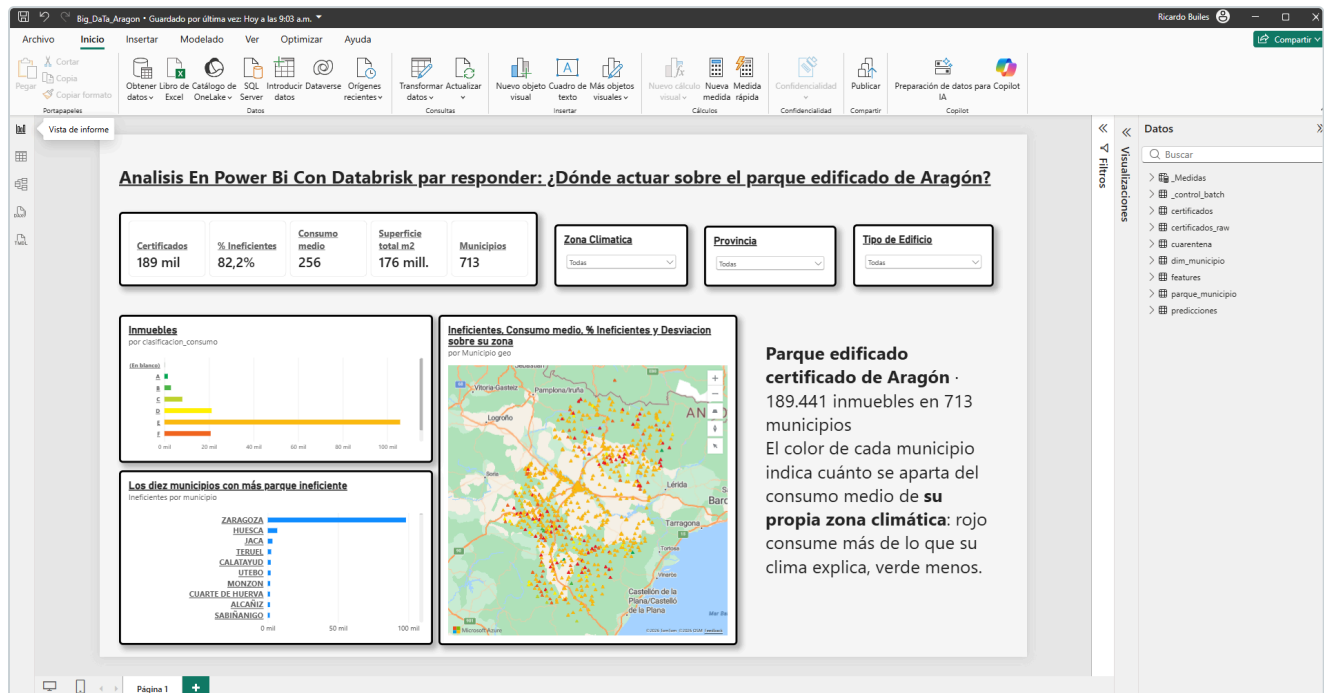


Figura 8.4 — Informe de Power BI sobre el modelo anterior, con los segmentadores aplicados a todas las tablas a través de la dimensión de municipio.

Tablero web

Una tercera superficie, independiente de las dos anteriores, recoge en tres vistas el estado del pipeline, la calidad del dato y la validación del modelo. Al ser autónoma, permite presentar los resultados sin depender de que el SQL Warehouse esté activo.

Exposición del modelo

El modelo queda registrado en MLflow, que es el paso previo al despliegue. Habilitarlo como API no requiere cambios en el código —es una acción de configuración sobre el modelo ya registrado— pero la edición de Databricks utilizada no permite crear serving endpoints. El consumo se resolvió por la vía de los tableros conectados a las tablas Gold.

Decisiones y correcciones

Esta sección recoge los errores que encontramos en nuestro propio pipeline y cómo se resolvieron. Los incluimos porque cada uno cambió una decisión de diseño, y porque el procedimiento que los destapó es replicable.

La regla de valores atípicos estaba mal calibrada

La primera versión de R06 rechazaba consumos por encima de «percentil 99 × 10», un criterio automático sin significado físico. Al comprobar la capa Silver encontramos **1.458 filas por encima de 1.000 kWh/m²·año** y un máximo de 384.328: la regla no filtraba nada.

El diagnóstico fue que el percentil usado para construir el umbral estaba a su vez contaminado por los valores extremos. Se sustituyó por un rango de dominio, [20, 1000], y se regeneró Silver desde Bronze.

MÉTRICA EN SILVER	ANTES	DESPUÉS
Máximo de consumo	384.328	996,6
Mínimo de consumo	0	20,0
Percentil 99	797,3	639,0
Filas en cuarentena	537	2.704

La primera matriz de confusión era degenerada

Para derivar la clasificación usamos inicialmente, como umbral de ineficiencia, el consumo mínimo de la letra E en cada zona. El resultado fue una matriz sin dos de sus cuatro casillas: **el 99,87 % de los inmuebles quedaba clasificado como ineficiente**, con recall del 100 % y ningún verdadero negativo.

La causa es que la letra E representa más de la mitad del parque y su rango se solapa con las demás: basta un inmueble con letra E y consumo bajo para hundir el umbral. Se sustituyó por el **percentil 95 del consumo de los edificios A–D** de cada zona, que define el techo del grupo eficiente en lugar del suelo del ineficiente.

INDICADOR	UMBRAL INICIAL	UMBRAL CORREGIDO
Tasa base de ineficientes	99,87 %	22,6 %
Verdaderos negativos	0	27.295
Precisión	99,9 %	59,2 %
Lift	1,00×	2,62×

Las métricas empeoran y el modelo mejora. Es el mismo fenómeno que la fuga de datos: un resultado demasiado bueno suele ser señal de que la medición está mal planteada.

La validación no era compatible con el cómputo serverless

El notebook de validación usaba la API de RDD, que Databricks bloquea en cómputo serverless. El job fallaba en la tarea 03 sin que el problema tuviera relación con los datos. Se reescribió con la API de DataFrame y se le añadió que, al fallar, informe de qué invariante no se cumple, cuántas filas la incumplen y cinco ejemplos.

```
antes      4 min 59 s  · fallo por API no permitida
después    10 s      · informe de qué invariante no se cumple
```

Las coordenadas del origen son inservibles

El origen trae las coordenadas UTM de cada inmueble en un solo campo de texto. Al separarlas, la componente Y quedó **íntegramente vacía**: 187.570 valores en X y ninguno en Y. Sin las dos componentes no hay punto, y aunque lo hubiera, las coordenadas UTM requieren reproyección para representarse en un mapa.

La ubicación quedó representada por provincia y zona climática.

El guion como valor ausente

La columna de calificación traía 76 filas con el valor . Un valor centinela no es una categoría: si se deja como texto, el motor lo trata como una octava letra de una escala de siete. Se normalizó a nulo dentro del propio pipeline, de forma que la corrección se mantiene aunque Silver se regenere.

Limitaciones

El dataset es un censo de trámites, no del parque. Solo contiene inmuebles que en algún momento se vendieron o alquilaron. El modelo aprende, por tanto, de una muestra sesgada hacia el mercado inmobiliario activo.

La zona climática es una aproximación. El Código Técnico de la Edificación la asigna por capital de provincia corrigiendo por altitud del municipio. Nuestra derivación no incorpora la corrección altitudinal completa, lo que introduce ruido.

No hay ubicación precisa. Al perderse una de las dos componentes de las coordenadas, la granularidad geográfica máxima es el municipio.

El recall es bajo. Con un 33,1 %, el modelo no sirve como único filtro para localizar todos los inmuebles ineficientes. Su utilidad está en priorizar, no en descartar.

El desempeño no es homogéneo entre zonas. El error aumenta y el lift disminuye en las zonas con menos observaciones, lo que desaconseja aplicar el resultado global de forma uniforme.

La tabla de predicciones es de evaluación, no de producción. Contiene los 37.801 inmuebles del conjunto de prueba, que sí tienen certificado. Un despliegue real requeriría puntuar el parque no certificado, y para eso haría falta cargar ese universo desde el catastro.

El modelo no está expuesto como API. Queda registrado en MLflow, que es el paso previo, pero la edición de Databricks utilizada no permite crear serving endpoints.

Qué haríamos con más tiempo

- **Enriquecer con el Catastro** para recuperar el año de construcción real de las filas que hoy se rechazan, en lugar de descartarlas
- **Separar el pipeline de inferencia** del de entrenamiento: tienen cadencias distintas, y reentrenar de forma automática sin control de deriva degradaría el modelo
- **Incorporar variables de la envolvente** del edificio, que es donde está la mayor parte de la varianza que el modelo no explica

Reproducibilidad

Estructura del proyecto

cee_aragonv2	catálogo de Unity Catalog
└─ landing/raw	volume · el CSV tal como llegó
└─ bronze/certificados_raw	192.145 filas · todo texto
└─ silver/certificados	189.441 válidas
└─ silver/cuarentena	2.704 rechazadas, con motivo
└─ silver/dim_municipio	municipios · zona climática
└─ gold/features	tabla de entrenamiento, sin fuga
gold/parque_municipio	agregado municipal
gold/predicciones	evaluación sobre el conjunto de prueba

Notebooks

01_Bronze_Ingesta_Certificados	ingesta y capa cruda
02_Calidad_Datos_Bronze	perfilado
03_Silver_Certificados	reglas, cuarentena, reconciliación
validate_silver_certificados	invariantes · corta el job si fallan
04_Zona_Climatica_Gold	dimensión y tablas de consumo
05_ML_Prediccion_Consumo	modelo y evaluación

Cómo reproducirlo

- Descargar el CSV del registro y subirlo al volume `landing/raw`
- Ejecutar `01_Bronze_Ingesta_Certificados`, que es idempotente y puede correrse tantas veces como se quiera
- Lanzar el job `CEE_Aragon_ETL_Pipeline`, que encadena las cuatro tareas en orden
- Ejecutar `05_ML_Prediccion_Consumo` para entrenar y evaluar

El modelo y sus métricas quedan registrados en MLflow, de modo que cualquier ejecución posterior es comparable con las anteriores.

Sobre el uso de asistentes de IA

Este proyecto se desarrolló usando asistentes de inteligencia artificial para la generación de código PySpark y SQL, y para la revisión de los textos. Las decisiones de diseño —qué variables admitir, dónde poner los umbrales, cómo tratar los valores ausentes, qué métrica reportar— se tomaron y justificaron por nosotros, y están documentadas en las secciones 5, 7 y 9.

Varias de las correcciones que recoge la sección 9 consistieron precisamente en **rechazar una solución propuesta por el asistente**: ampliar el umbral de validación para que el job pasara, en lugar de corregir la regla de limpieza que lo provocaba. Lo señalamos porque coincide con lo planteado en clase: en un flujo de trabajo asistido, el criterio para validar lo que el asistente produce es la parte que no se delega.

